

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Práctica:

SmartInvoice

PONDERACIÓN: 10 pts

 **Tiempo estimado: 15 hrs/min**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Habilidad(es) blandas a formar.....	3
2. Resultado del Aprendizaje.....	3
2.1. Objetivo SMART.....	3
3. Enunciado de la Práctica.....	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance de la práctica.....	5
Restricciones.....	6
Lenguajes y herramientas.....	6
Requisitos mínimos del sistema.....	7
4. Entregables.....	8
5. Material de apoyo.....	9
6. Recursos y herramientas a utilizar.....	10
Software / Hardware.....	10
Plataformas.....	11

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	El estudiante desarrolla soluciones inteligentes capaces de procesar información de manera automática, garantizando la integridad de los datos extraídos, la correcta ejecución de los procesos automatizados y la generación de resultados confiables para la toma de decisiones dentro de una organización.

1.2. Competencia(s)

Diseñar e implementar soluciones inteligentes que integren técnicas de Computer Vision, Reconocimiento Óptico de Caracteres (OCR) y Automatización Robótica de Procesos (RPA), mediante aplicaciones web, APIs REST, bases de datos y servicios backend, para automatizar el procesamiento, análisis y gestión de documentos digitales en entornos organizacionales.

1.3. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

- La práctica le permitirá desarrollar las siguientes habilidades:
- Pensamiento lógico y analítico.
- Resolución de problemas.
- Atención al detalle.
- Capacidad de investigación.
- Organización y estructuración de soluciones.
- Trabajo autónomo y proactividad.
- Pensamiento crítico para la automatización y análisis de información.
- Capacidad de integración de tecnologías.
- Toma de decisiones basada en datos.
- Adaptabilidad ante problemas tecnológicos complejos.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Desarrollar una plataforma inteligente capaz de procesar documentos	El sistema deberá procesar al menos tres tipos distintos de documentos, extraer información relevante,	Implementan do un backend en Python, una API REST, una base de	Para fortalecer las competencias relacionadas con Inteligencia Artificial,	Al finalizar la práctica y entregar el proyecto en la fecha establecida.

digitales mediante técnicas de Computer Vision y OCR, automatizando tareas administrativas utilizando RPA.	almacenarla en una base de datos, generar reportes automáticos y ejecutar procesos RPA asociados.	datos, una interfaz web administrativa, herramientas OCR, automatización frontend y despliegue en la nube.	Computer Vision, Automatización de Procesos, integración de sistemas y desarrollo de aplicaciones empresariales modernas.	
--	---	--	---	--

3. Enunciado de la Práctica

3.1 Descripción del problema a resolver

Las empresas reciben diariamente una gran cantidad de facturas en formatos digitales como imágenes, documentos PDF o archivos escaneados. El procesamiento manual de estas facturas implica revisar cada documento, identificar la información relevante, registrar los datos en los sistemas internos, generar reportes administrativos y notificar a los responsables del proceso.

Este procedimiento suele consumir una cantidad considerable de tiempo y es propenso a errores humanos, especialmente cuando el volumen de documentos aumenta. Como consecuencia, las organizaciones enfrentan retrasos en sus procesos administrativos, inconsistencias en la información registrada y dificultades para mantener un control eficiente de los documentos procesados.

Con el objetivo de optimizar estas actividades, la empresa ha decidido implementar una solución inteligente denominada **SmartInvoice**, la cual integrará técnicas de Computer Vision, Reconocimiento Óptico de Caracteres (OCR) y Automatización Robótica de Procesos (RPA) para automatizar el procesamiento de facturas.

El sistema deberá permitir la carga de facturas en formato imagen o PDF, identificar automáticamente la información contenida en cada documento mediante técnicas de visión por computadora y OCR, extraer datos relevantes como número de factura, fecha, proveedor, NIT, subtotal, impuestos y monto total, y almacenarlos en una base de datos para su posterior consulta.

Posteriormente, el sistema deberá ejecutar procesos automatizados que permitan validar la información extraída, generar reportes administrativos, mantener una bitácora de procesamiento y notificar automáticamente los resultados obtenidos mediante correo electrónico.

La solución deberá proporcionar una interfaz web que permita a los usuarios cargar documentos, consultar información procesada, visualizar reportes y monitorear el estado de las automatizaciones ejecutadas.

3.2 Alcance de la práctica

La práctica deberá incluir como mínimo los siguientes componentes:

Obligatorios

- Implementación de un backend utilizando Python.
- Desarrollo de una API REST para la comunicación entre el frontend, la base de datos y los módulos de procesamiento inteligente.
- Implementación de una base de datos para el almacenamiento de facturas, proveedores, usuarios, bitácoras y resultados del procesamiento.
- Desarrollo de una interfaz web administrativa para la carga, consulta y gestión de facturas.
- Implementación de un mecanismo de autenticación para el acceso al sistema.
- Implementación de operaciones CRUD para la administración de proveedores.
- Gestión de facturas procesadas
- Implementación de una bitácora de ejecución que almacene como mínimo:
 - Fecha y hora de procesamiento.
 - Usuario responsable.
 - Documento procesado.
 - Estado del procesamiento.
 - Resultado obtenido.
- Carga de facturas en formato PDF, JPG, JPEG o PNG.
- Implementación de técnicas de Computer Vision y OCR para la extracción automática de información contenida en las facturas.
- Extracción automática de al menos los siguientes campos:
 - Número de factura.
 - Fecha.
 - Nombre del proveedor.
 - NIT.
 - Subtotal.
 - Impuestos.
 - Total de la factura.
- Almacenamiento automático de la información extraída en la base de datos.
- Implementación de una validación automática de los datos extraídos antes de su almacenamiento definitivo.
- Generación automática de reportes administrativos en formato PDF, Excel o CSV.
- Implementación de una automatización RPA que permita registrar automáticamente la información extraída en formularios web o sistemas simulados.
- Implementación de una automatización para el envío de reportes mediante correo electrónico.
- Implementación de Docker para la contenedorización de la solución.
- Implementación de Docker Compose para la ejecución de los servicios del proyecto.
- Despliegue funcional de la solución en un proveedor de nube de libre elección.
- Publicación de una URL accesible para la evaluación del proyecto.
- Uso de control de versiones mediante Git y repositorio en GitHub.
- Documentación de instalación, ejecución y despliegue del sistema.

Opcional

- Dashboard con métricas de procesamiento.
- Clasificación automática de facturas por categoría o proveedor.
- Procesamiento masivo de múltiples facturas simultáneamente.
- Detección automática de facturas duplicadas.
- Validación automática de formatos de NIT.
- Exportación de reportes a múltiples formatos.
- Visualización gráfica de estadísticas.
- Programación automática de tareas mediante scheduler.
- Integración con APIs externas.
- Implementación de pruebas automatizadas.
- Procesamiento en segundo plano mediante colas de tareas.

Restricciones

- El backend deberá desarrollarse exclusivamente utilizando Python.
- Las funcionalidades de Computer Vision y OCR deberán ser implementadas por el estudiante utilizando librerías compatibles con Python.
- No se permite utilizar servicios externos que realicen completamente la extracción de información mediante inteligencia artificial generativa.
- Todas las facturas deberán procesarse localmente dentro de la solución desarrollada.
- La información extraída deberá almacenarse en una base de datos.
- El sistema deberá implementar obligatoriamente OCR y al menos una automatización RPA funcional.
- El proyecto deberá ejecutarse mediante Docker Compose.
- El proyecto deberá estar desplegado en la nube al momento de la evaluación.
- La arquitectura del sistema será de libre elección; sin embargo, deberá documentarse y justificarse en el manual técnico.
- El código fuente deberá mantenerse en un repositorio GitHub accesible para evaluación.
- Durante la evaluación presencial, el estudiante deberá demostrar el procesamiento completo de una factura desde su carga hasta la generación del reporte final.

3.3 Requerimientos técnicos

Para el desarrollo de esta práctica se deberán utilizar las siguientes tecnologías:

Lenguajes y herramientas

Para el desarrollo de esta práctica se deberán utilizar las siguientes tecnologías:

- Python 3.x para el desarrollo del backend, los módulos de Computer Vision, OCR y las automatizaciones RPA.
- Framework web de libre elección (FastAPI, Flask o equivalente).

- HTML, CSS y JavaScript para el desarrollo del frontend.
- Base de datos de libre elección (PostgreSQL, MySQL, SQLite, MongoDB o equivalente).
- Librerías de Computer Vision como OpenCV o equivalentes.
- Librerías OCR como Tesseract OCR, EasyOCR o equivalentes.
- Librerías para automatización frontend como Selenium, Playwright o equivalentes.
- Librerías para generación de reportes en formato PDF, Excel o CSV.
- Librerías para envío de correos electrónicos mediante SMTP o APIs equivalentes.
- Docker para la creación de contenedores.
- Docker Compose para la orquestación de servicios.
- Git para el control de versiones.
- GitHub para el almacenamiento del repositorio.

Requisitos mínimos del sistema

- Sistema de autenticación funcional.
- Panel administrativo accesible desde navegador web.
- Base de datos configurada y conectada al sistema.
- API REST funcional para la comunicación entre los componentes del sistema.
- Carga de facturas en formato PDF, JPG, JPEG o PNG.
- Procesamiento automático de facturas mediante OCR.
- Extracción automática de información relevante contenida en las facturas.
- Registro de la información extraída dentro de la base de datos.
- Administración de proveedores mediante operaciones CRUD.
- Administración de facturas procesadas mediante operaciones de consulta y visualización.
- Bitácora de procesamiento que permita consultar el historial de documentos analizados.
- Generación automática de reportes administrativos utilizando la información almacenada en el sistema.
- Automatización RPA para el registro automático de información en formularios web o sistemas simulados.
- Automatización para el envío de reportes mediante correo electrónico.
- Visualización de resultados obtenidos durante el procesamiento de cada factura.
- Registro del estado de procesamiento de cada documento (Procesado, Pendiente, Error o Rechazado).
- Proyecto ejecutable mediante Docker Compose.
- Proyecto desplegado en la nube y accesible mediante una URL pública.
- Documentación de instalación, ejecución y despliegue.
- Repositorio GitHub con historial de cambios del proyecto.
- El sistema deberá ser capaz de procesar correctamente al menos 20 facturas de prueba durante la evaluación.

4. Entregables

Tipo	Descripción
Repositorio del Proyecto	Enlace al repositorio GitHub que contenga el código fuente, documentación, historial de cambios y evidencias del desarrollo.
Backend	Proyecto desarrollado en Python encargado del procesamiento OCR, Computer Vision, automatizaciones RPA, lógica de negocio y comunicación con la base de datos.
API REST	Servicio REST funcional para la administración de usuarios, proveedores, facturas, reportes y bitácoras del sistema.
Frontend	Interfaz web administrativa que permita cargar facturas, consultar información procesada, visualizar reportes y monitorear el estado de las automatizaciones.
Base de Datos	Modelo de datos implementado y configurado para el almacenamiento de usuarios, proveedores, facturas, bitácoras y reportes generados por el sistema.
Módulo OCR	Implementación funcional del proceso de extracción automática de texto a partir de imágenes o documentos PDF.
Módulo Computer Vision	Implementación funcional de técnicas de visión por computadora para el procesamiento y análisis de documentos digitales.
Automatización RPA	Implementación funcional de al menos una automatización RPA que registre información extraída en formularios web o sistemas simulados.
Generación de Reportes	Implementación funcional para la generación automática de reportes administrativos en formato PDF, Excel o CSV.
Envío Automático de Correos	Implementación funcional del envío automático de reportes por correo electrónico.
Bitácora de Procesamiento	Registro histórico de las facturas procesadas, errores detectados, resultados obtenidos y automatizaciones ejecutadas.
Requerimientos Funcionales	Estas irán en el Manual Técnico. Describan las funcionalidades implementadas dentro del sistema.
Requerimientos No Funcionales	Estas irán en el Manual Técnico en formato Markdown (.md) y describirán aspectos relacionados con rendimiento, seguridad, mantenibilidad,

	disponibilidad y usabilidad.
Patrón de Arquitectura	Deberá incluir un diagrama que represente la arquitectura implementada y la interacción entre los componentes del sistema.
Manual Técnico	Documento en formato Markdown (.md) que describa la arquitectura implementada, tecnologías utilizadas, módulos OCR, Computer Vision, RPA, API REST, base de datos, despliegue y posibles mejoras futuras.

5. Material de apoyo

Se recomienda consultar los siguientes recursos para comprender los algoritmos de búsqueda, el desarrollo de APIs REST y la construcción de aplicaciones web para la visualización de resultados.

Documentación oficial

- Python:
<https://docs.python.org/3>
- FastAPI:
<https://fastapi.tiangolo.com>
- Flask:
<https://flask.palletsprojects.com>
- OpenCV:
<https://docs.opencv.org>
- Tesseract OCR:
<https://tesseract-ocr.github.io>
- EasyOCR:
<https://www.jaided.ai/easyocr>
- Selenium:
<https://www.selenium.dev/documentation>
- Playwright:
<https://playwright.dev/python>
- Docker:
<https://docs.docker.com>
- Docker Compose:
<https://docs.docker.com/compose>
- Git:
<https://git-scm.com/doc>
- GitHub:
<https://docs.github.com>

Recursos sobre Computer Vision y OCR

- Introducción a Computer Vision:
<https://opencv.org>

- OCR con Tesseract:
<https://tesseract-ocr.github.io/tessdoc>
- OCR con EasyOCR:
<https://www.jaided.ai/easyocr>
- Procesamiento de imágenes con OpenCV:
https://docs.opencv.org/master/d6/d00/tutorial_py_root.html
- Extracción de texto desde imágenes:
<https://realpython.com/ocr-a-practical-introduction-to-optical-character-recognition-in-python>

Recursos sobre Automatización Robótica de Procesos (RPA)

- Introducción a RPA:
https://en.wikipedia.org/wiki/Robotic_process_automation
- Automatización Web con Selenium:
<https://selenium-python.readthedocs.io>
- Automatización Web con Playwright:
<https://playwright.dev/python/docs/intro>

Recursos sobre generación de reportes y correos electrónicos

- ReportLab:
<https://docs.reportlab.com>
- FPDF2:
<https://py-pdf.github.io/fpdf2>
- OpenPyXL:
<https://openpyxl.readthedocs.io>
- Envío de correos electrónicos con Python:
<https://realpython.com/python-send-email>

6. Recursos y herramientas a utilizar

Software / Hardware

- Computadora personal con sistema operativo Windows, Linux o macOS.
- Python 3.11 o superior.
- Visual Studio Code o cualquier IDE equivalente.
- Docker Desktop o Docker Engine.
- Docker Compose.
- Git para control de versiones.
- Navegador web actualizado.
- Cliente para pruebas de APIs (Postman, Insomnia o equivalente).
- Base de datos de libre elección (PostgreSQL, MySQL, SQLite, MongoDB o equivalente).

Plataformas

- GitHub para alojamiento del repositorio y control de versiones.
- Proveedor de nube de libre elección para el despliegue del sistema (Render, Railway, Azure, AWS, GCP o equivalente).
- UEDI para la entrega de la práctica.

Se utilizará el mismo repositorio del curso

Fecha de entrega: 19/06/2026